FACULTAD DE INFORMÁTICA
MÁSTER DE INGENIERÍA DEL SOFTWARE



SolidarianID

Memoria de prácticas

Grupo 3:
Antonio Pérez Serrano
Fabián Sola Durán
Manuel Bernabé Rodríguez
Pablo Peris Solano
Sergio Ros Liarte

Desarrollo Full-Stack
30/04/2026

Índice

1. Introducción **3**

2. Arquitectura general de la solución **3**
2.1. Aplicación principal (MVP) basada en microservicios 3
2.2. Aplicación de administración: monolito MVC con SSR 4

3. Frontend **5**
3.1. App Router 6
3.2. Renderizados de Next.js 6
3.2.1. Renderizado SSG 6
3.2.2. Renderizado CSR 8
3.2.3. Renderizado combinado (SSR + CSR) 9
3.3. Redux: Contexto de autenticación 10
3.3.1. Estructura del Store y Tipado 10
3.3.2. El Slice de Autenticación 11
3.3.3. Inicialización del Contexto 12
3.3.4. Acceso a los datos desde componentes 13
3.4. Cliente de GraphQL con apollo 14
3.4.1. Configuración centralizada y entorno centrado en el polimorfismo 14
3.4.2. Gestión de errores transversal (ErrorLink) 15
3.4.3. Capa de Abstracción para Consultas (`GraphqlClientService`) . . 16
3.4.4. Uso del cliente genérico en servidor (SSR) en Home 17
3.4.5. Uso en cliente con hooks de Apollo: `useQuery` y `useMutation` . . 18
3.4.6. Suscripciones en ventana de notificaciones (`useSubscriptions`) . 20
3.5. Librerías de componentes utilizadas 22
3.5.1. Shadcn/ui (núcleo de la interfaz). 22
3.5.2. Comparativa con React Bootstrap y justificación de descarte . . . 23
3.6. PWA en SolidarianID 23
3.6.1. Fichero `manifest` 23
3.6.2. Service worker, estrategias de caché y app shell 24
3.6.3. Notificaciones push 26
3.7. Almacenamiento local basado en Dexie 29
3.7.1. Recibir y guardar notificaciones en Dexie 29
3.7.2. Leer notificaciones de Dexie 31

4. Conclusiones **32**

A. Nuestra interfaz **35**
A.1. Páginas de la interfaz 35
A.2. Componentes interesantes de la interfaz 45

Índice de figuras

1. Arquitectura de contenedores (C2) del MVP de SolidarianID: microservicios de *Users* y *Communities* y aplicación de administración monolítica. . 4
2. App router 7
3. Landing page 35
4. Inicio de sesión 35
5. Registro de usuarios 36
6. Preferencias 36
7. Home page 37
8. Explorar comunidades y causas 37
9. Formulario de creación de la comunidad 38
10. Formulario de creación de una causa 38
11. Formulario de creación de una acción 39
12. Detalle de comunidad 39
13. Detalle de causa 40
14. Detalle de acción 40
15. Mis causas 41
16. Notificaciones del usuario 41
17. Mi perfil 42
18. Mis solicitudes 42
19. Mi información personal 43
20. Centro de soporte y ayuda 43
21. Mis favoritos 44
22. Botón de compartir 45

1. Introducción

En esta memoria abarcaremos los puntos más importantes del diseño y la implementación software de SolidarianID, el MVP de la aplicación solidaria que engloba el desarrollo de las prácticas del segundo cuatrimestre del Máster en Ingeniería del Software. En concreto, ahondaremos en la arquitectura general que hemos diseñado para la aplicación y los detalles más importantes respecto al desarrollo de su interfaz gráfica.

En la bibliografía se detalla el enlace [1] a nuestro repositorio de *GitHub* en el cual se pueden encontrar las distintas carpetas de código fuente que hemos llevado a cabo.

2. Arquitectura general de la solución

En el MVP de *SolidarianID* hemos diseñado una arquitectura compuesta por **tres aplicaciones principales** con responsabilidades claramente diferenciadas: (i) la **aplicación principal** basada en **microservicios** (expuesta por un API REST), (ii) la **aplicación de administración** implementada como un **monolito MVC** que integra **backend y frontend** mediante *Server-Side Rendering* (SSR) y (iii) finalmente la aplicación frontend principal construida en *Next.js*, siendo esta última la que se ha añadido este segundo cuatrimeste. Esta separación nos permite evolucionar el *core* funcional de la plataforma de forma independiente, manteniendo además límites claros de acceso y despliegue.

2.1. Aplicación principal (MVP) basada en microservicios

La aplicación principal del MVP se implementa como un conjunto de **APIs independientes** desarrolladas con NestJS y organizadas en un *monorepo*, donde cada aplicación representa un contexto delimitado del dominio. En particular, el MVP expone:

- **Microservicio de usuarios** (`apps/users/`): encargado del registro, autenticación y gestión del perfil de usuario. Expone una API REST (documentada con Swagger) y publica eventos de dominio cuando se producen cambios relevantes.
- **Microservicio de comunidades** (`apps/communities/`): responsable de la gestión de comunidades y del contenido asociado (causas, acciones, eventos, objetivos), así como del flujo de solicitudes de creación que serán revisadas desde administración.

Desde el punto de vista de interacción, los usuarios acceden a la plataforma mediante la aplicación *frontend* construida en ***Next.js***. Tal como se ilustra en la Figura 1, la comunicación de este *frontend* con la aplicación principal se realiza a través de un *proxy* inverso de *Nginx*, que redirige las peticiones al microservicio correspondiente. Además, para permitir esta comunicación se ha activado la configuración de CORS tanto en el microservicio de usuarios como en el de comunidades, ya que estos son los que consume el frontend de *Next.js* para extraer la información.

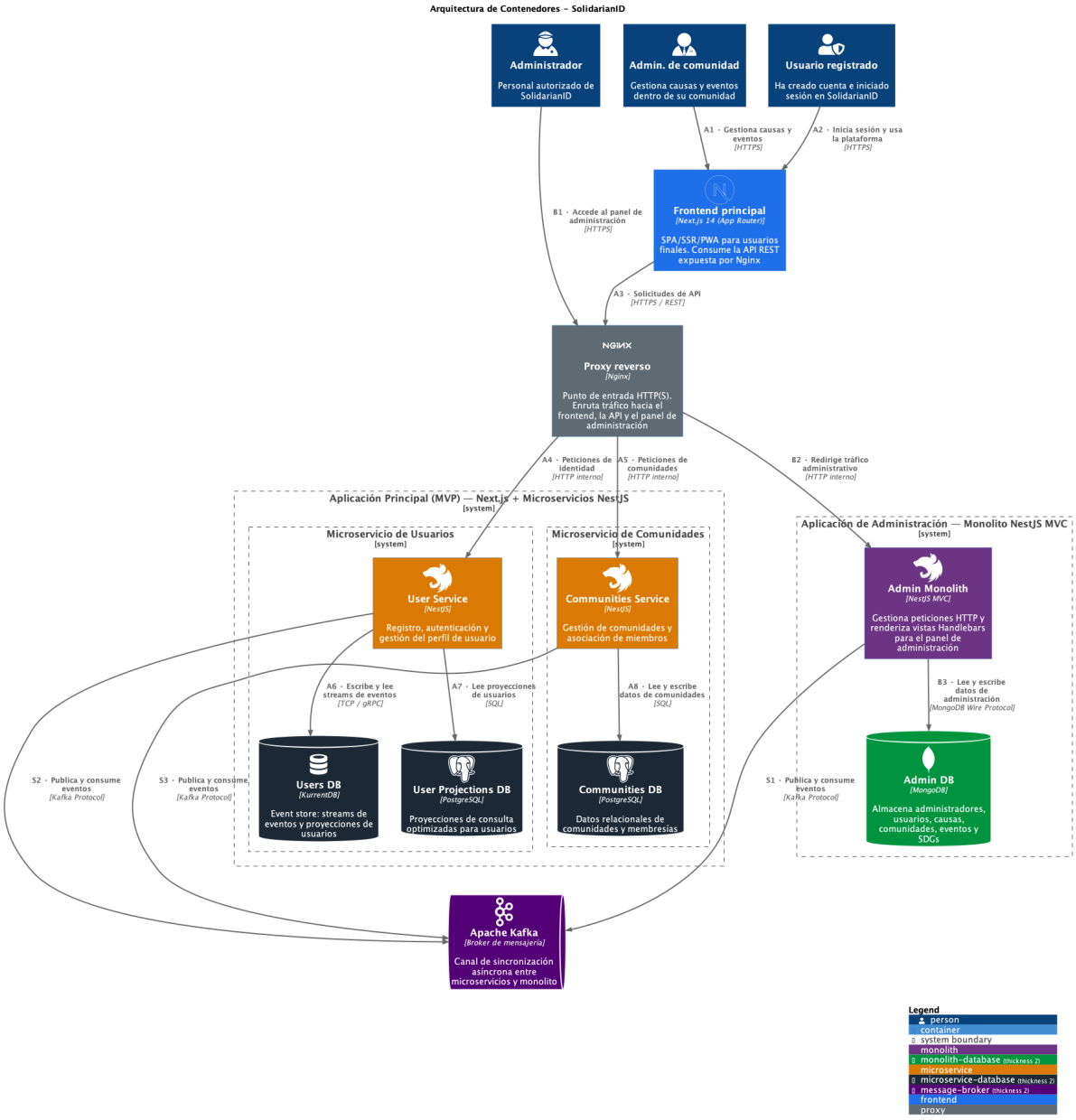


Figura 1: Arquitectura de contenedores (C2) del MVP de SolidarianID: microservicios de *Users* y *Communities* y aplicación de administración monolítica.

2.2. Aplicación de administración: monolito MVC con SSR

En paralelo, se ha desarrollado una **aplicación de administración** como un **monolito NestJS** con patrón **MVC**, orientado a las necesidades del **administrador de la plataforma**: moderación y validación de solicitudes, auditoría e informes, y consulta de métricas en el dashboard.

A diferencia de los microservicios (centrados en API), este monolito integra **backend y frontend** en la misma aplicación. El **renderizado de vistas** se realiza mediante:

- **Express** + **NestJS** como servidor HTTP.

- **Handlebars** como motor de plantillas para generar las vistas en el servidor (SSR).
- Estructura de vistas organizada en `views/` (layouts/pages/partials) y recursos estáticos en `public/` (CSS/JS/iconos).

Este enfoque resulta especialmente adecuado para un panel administrativo, ya que simplifica el despliegue (una única aplicación), centraliza la navegación y permite controlar el acceso mediante mecanismos de sesión de Express para administradores.

Tal y como se recoge en la memoria del proyecto, la comunicación entre componentes se apoya en **eventos** para sincronizar información relevante entre contextos (por ejemplo, solicitudes o cambios de usuarios) y evitar dependencias estrictamente síncronas entre servicios, manteniendo así un mayor desacoplamiento y escalabilidad evolutiva de los aplicativos por separado. Por otro lado, en la carpeta raíz del repositorio se encontrarán las consultas *Postman* asociadas para facilitar el testeo de la aplicación.

3. Frontend

En esta sección se detallan las funcionalidades implementadas en la interfaz de *SolidarianID*. La interfaz ha sido desarrollada en su totalidad con el *framework* ***Next.js***, ya que este aporta una gran variedad de funcionalidades que permiten enriquecer el código de la interfaz, tales como el enrutamiento ***App Router*** y los distintos **tipos de renderizado** (*SSG, SSR, CSR, etc*). Durante los próximos apartados se detalla con precisión el *stack* tecnológico utilizado y las funcionalidades más importantes de la aplicación *frontend*.

Librerías utilizadas

En este apartado se comentará de forma genérica el *stack* de librerías utilizadas y el propósito de añadirlas a nuestro proyecto *frontend* de *SolidarianID*. Las librerías principales que se explican en este documento son las siguientes:

- **Next**: *Framework* sobre el cual se ha construido la aplicación *React*, con el fin de aprovechar sus funcionalidades de enrutamiento (app router), bundler (*turbopack*) y distintos tipos de renderizado.
- **React Redux**: Librería utilizada para guardar el contexto global del usuario autenticado en la *app*.
- **Apollo client**: Librería utilizada para las peticiones *GraphQL* .
- **Shadcn/ui**: Librería de componentes basada en *Base UI*.
- **Serwist**: Librería de gestión del *service worker* para soporte PWA y definir estrategias de caché.
- **Dexie**: Librería de almacenamiento local de datos basada en *IndexDB* y utilizada para guardar las notificaciones provenientes de eventos *graphql*.

Otras librerías que también han sido de gran ayuda para el desarrollo del proyecto son las siguientes:

- **Tabler UI Icons**: Librería de iconos.
- *Tailwind CSS*: Librería para estilar los componentes de la interfaz.
- **Motion**: Librería de movimiento de contenedores para dar dinamismo a la interfaz.
- **Zod**: Librería de validación de formularios en tiempo real, tales como los de login y registro.
- **Sonner**: Librería de notificaciones flotantes para proporcionar *feedback* visual al usuario.
- **Axios**: Librería utilizada para peticiones HTTP.
- **React select-country-list**: Librería de lista de países para seleccionar en el formulario de registro.
- **React phone-input-2**: Librería de prefijos telefónicos para el formulario de registro.

3.1. App Router

El indicador principal de que este proyecto utiliza *App router* es la presencia del directorio `app` principal para enrutar la aplicación. Tal como se puede observar en la Figura 2, se observa que desde el directorio app descienden dos subdirectorios: `(landing)` para la página de bienvenida y `(solidarian)` para las páginas que contienen la lógica de la aplicación. Los subdirectorios con paréntesis indican grupos de rutas con el fin de organizar el código, ya que estos no se incluyen en la URL, por tanto, la página raíz '/' corresponde con la *landing page*. Por otro lado, en la carpeta `(auth)` se pueden observar las páginas `page.tsx` dentro de `login` y de `register`, lo que indica que en las rutas `/login` y `/register` sí se renderiza contenido.

3.2. Renderizados de Next.js

En esta sección se detallan los distintos tipos de renderizado que se han utilizado durante el desarrollo de la aplicación, gracias a las posibilidades que ofrece el *framework Next.js*.

3.2.1. Renderizado SSG

La *landing page* utiliza SSG porque contiene únicamente contenido informativo estático que no requiere datos dinámicos ni personalizados. Esto maximiza la *performance* (entrega de contenido precompilado desde CDN), mejora el SEO, y reduce significativamente la

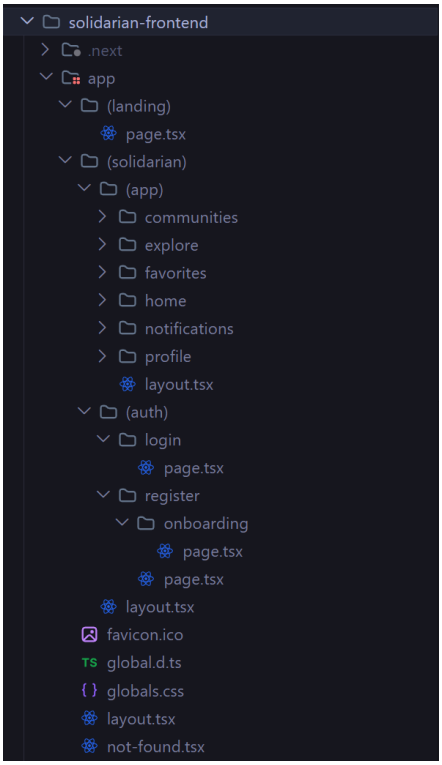


Figura 2: App router

carga del servidor. Esta es la estrategia recomendada en *Next.js* para páginas que no necesitan actualizarse frecuentemente. Tal como se puede observar en el trozo de código 1, sabemos que el componente `LandingPage` es SSG porque no es asíncrono y tampoco tiene la directiva 'use client'. Además, sabemos que el contenido se genera de forma estática al observar el *layout* padre mostrado en el trozo de código 2 (únicamente renderiza *children*, que en este caso sería la *landing page*).

```
import { Button } from '@/components/ui';
import { IconBolt, ... } from '@tabler/icons-react';
import Image from 'next/image';
import Link from 'next/link';

export default function LandingPage() {
  return (...);
}
```

Código 1: Renderizado SSG

```tsx
import { cn } from '@/lib/utils';
import { Geist_Mono, Inter } from 'next/font/google';
import './globals.css';

const inter = Inter({ subsets: ['latin'], variable: '--font-sans' });

const fontMono = Geist_Mono({
  subsets: ['latin'],
  variable: '--font-mono',
});

export default function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode;
}>) {
  return (
    <html
      lang="en"
      suppressHydrationWarning
      className={cn('antialiased', ...)}
    >
      <body>{children}</body>
    </html>
  );
}
```

Código 2: Layout principal

3.2.2. Renderizado CSR

Un ejemplo de página CSR en la aplicación es la página de *login*. Esta debe ser CSR porque combina validación en tiempo real de los campos del formulario (gracias a la librería *zod*) con el uso de algunos *hooks* para manejar interactividad (toggle de contraseña (*showPassword*) , *isLoading* para la comunicación con el servidor, etc), tal como se puede apreciar en la Figura 3. Además se puede observar que el fichero comienza con la directiva `'use client';`, lo que nos indica claramente que el componente es CSR. En este caso CSR es la estrategia correcta: renderiza la UI localmente, valida los campos en el cliente, y solo comunica con el servidor en el submit final (autenticación).

```
'use client';

import { Button, ... } from '@/components/ui';
...

export default function LoginPage() {
  const [isLoading, setIsLoading] = useState(false);
  const [showPassword, setShowPassword] = useState(false);
  const dispatch = useDispatch();
  const router = useRouter();
  ...

  return (...);
}
```

Código 3: Renderizado CSR

3.2.3. Renderizado combinado (SSR + CSR)

La página de Inicio `HomePage` de *SolidarianID* combina renderizado en el servidor y en el cliente para equilibrar rendimiento e interactividad. En *Next.js*, el componente principal de la página se ejecuta en el servidor porque es asíncrono y no declara `'use client'`:

```
export default async function HomePage() {
  noStore();

  const { communities, causes, errorCommunities, errorCauses } = await loadHomeData();
  const styles = homeStyles;

  return (
    <div className={styles.container}>
      ...
    </div>
  );
}
```

Código 4: Renderizado híbrido SSR + CSR

La función `loadHomeData` resuelve los datos antes de pintar la interfaz. Primero consulta las comunidades y las causas con *Apollo*, y después completa cada elemento con una imagen obtenida desde *Pexels*:

```
const [communitiesResult, causesResult] = await Promise.allSettled([
  graphql.query(GET_COMMUNITIES_QUERY, {}),
  graphql.query(GET_CAUSES_QUERY, {}),
]);

if (communityList.length > 0) {
  const imageService = new PexelsImageService();
  communities = await Promise.all(
    communityList.map(async community => {
      const banner = await imageService.getDeterministicBanner({ uuid: community.id,
      ↪  theme: community.name });
      return { ...community, imageUrl: banner.large };
    })
  );
}
```

Con esto, *Next.js* genera HTML inicial con contenido real en el servidor. Después, los componentes del cliente se hidratan en el navegador para aportar estado, eventos y navegación interactiva. Un ejemplo es el componente `HeaderHome` que se usa dentro de la página principal. Además, el *layout* de la aplicación también se ejecuta en cliente y proporciona los proveedores globales necesarios para la experiencia interactiva, como *Redux*, *Apollo* y el tema visual. En conjunto, la página usa SSR para entregar contenido inicial rápido y CSR para completar la interacción dinámica.

3.3. Redux: Contexto de autenticación

En esta sección se detallará como se ha utilizado la librería ***React Redux*** para gestionar el contexto de autenticación del usuario.

3.3.1. Estructura del Store y Tipado

La gestión del contexto de usuario se centraliza en *Redux* a través del `store.ts`, que configura el almacén global con *Redux Toolkit* (`configureStore`). Dicha implementación se puede observar en el trozo de código 5. Además, se definen dos tipos fundamentales: `RootState` representa la estructura completa del estado global (que incluye la propiedad *auth* con los datos del usuario) y `AppDispatch` es el tipo de la función *dispatch*.

```ts
import { authSlice } from '@/features/redux/userSlice';
import { configureStore } from '@reduxjs/toolkit';

const store = configureStore({
  reducer: {
    auth: authSlice.reducer,
  },
});

export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;
export default store;
```

Código 5: Fichero store.ts

Para garantizar *type-safety* en toda la aplicación, se crean *hooks* personalizados en el fichero `redux-hook.ts` (`useAppSelector` y `useAppDispatch`) que ya vienen tipados automáticamente, evitando tener que especificar los tipos manualmente cada vez que se usan. Esto se puede observar en el trozo de código 6.

```ts
import type { AppDispatch, RootState } from '@/features/redux/store';
import { useDispatch, useSelector } from 'react-redux';

export const useAppSelector = useSelector.withTypes<RootState>();
export const useAppDispatch = useDispatch.withTypes<AppDispatch>();
```

Código 6: Fichero redux-hook.ts

3.3.2. El Slice de Autenticación

El núcleo de la gestión del usuario está en `userSlice.ts`, un *slice* de *Redux Toolkit* que define la interfaz `UserState` con una propiedad `currentUser` que puede ser un usuario o null. En el trozo de código 7 se muestra el modelo del usuario que se pasa al contexto de la aplicación.

```ts
export interface User {
  id: string;
  username: string;
  fullName: string;
  mobilePhone: string;
  city: string;
  countryCode: string;
  preferences: {
      preferredSDGs: OdsId[];
      preferredCategories: CauseKey[];
      durationPreferences: DurationKey[];
      notificationsEnabled: boolean;
  };
}
```

Código 7: Modelo de usuario

Por otro lado, en el trozo de código 8 se muestra el *slice* que incluye tres funciones (*reducers*) para manejar las principales operaciones: `handleLogin` establece el usuario actual cuando se autentica (al hacer login exitoso), `updateCurrentUser` permite actualizar los datos del usuario (en la pantalla de mi perfil), y `handleLogout` limpia el estado al desconectarse (al cerrar sesión). Estas acciones se exportan para ser despachadas desde cualquier componente.

```
import { User } from '@/features/auth/model/user';
import { createSlice, type PayloadAction } from '@reduxjs/toolkit';

interface UserState {
  currentUser: User | null;
}

export const authSlice = createSlice({
  name: 'auth',

  initialState: {
    currentUser: null,
  } as UserState,

  reducers: {
    handleLogin: (state, action: PayloadAction<User>) => {
      state.currentUser = action.payload;
    },
    updateCurrentUser: (state, action: PayloadAction<Partial<User>>) => {
      if (!state.currentUser) {
        return;
      }

      state.currentUser = {
        ...state.currentUser,
        ...action.payload,
      };
    },
    handleLogout: state => {
      state.currentUser = null;
    },
  },
});

export const { handleLogin, ... } = authSlice.actions;
```

Código 8: Fichero userSlice.ts

3.3.3. Inicialización del Contexto

Cuando la aplicación carga, el componente `AuthInitializer.tsx` se encarga de restaurar el contexto de usuario. Este componente usa `useEffect` para llamar a `getCurrentUser()` al montar, por lo que obtiene los datos del usuario actualmente autenticado despachando la acción `handleLogin` para actualizar el estado de *Redux*. De esta forma, aunque el usuario recargue la página, su sesión se recupera automáticamente desde el *backend*. Dicha implementación se muestra en el trozo de código 9.

```
'use client';

import { User } from '@/features/auth/model/user';
import { authService } from '@/features/auth/services/auth.service';
import { handleLogin } from '@/features/redux/userSlice';
import { useEffect } from 'react';
import { useDispatch } from 'react-redux';

export function AuthInitializer({ children }: { children: React.ReactNode }) {
  const dispatch = useDispatch();

  useEffect(() => {
    async function initializeAuth() {
      const user: User = await authService.getCurrentUser();

      if (!user) return;

      dispatch(handleLogin(user));
    }

    initializeAuth();
  }, []);

  return children;
}
```

Código 9: Componente `AuthInitializer`

3.3.4. Acceso a los datos desde componentes

Cualquier componente puede acceder al estado *Redux* del usuario actual autenticado usando `useAppSelector(state =>state.auth.currentUser)`. También se pueden despachar acciones usando `useAppDispatch()` para modificar el estado gracias a las funciones de *login*, *logout* o *updateUser*. Este patrón centralizado garantiza que todos los cambios en el contexto de usuario pasen por *Redux*, manteniendo la aplicación predecible y centralizada, además de permitir que múltiples componentes reaccionen a cambios de autenticación de forma automática. En el trozo de código 10 se puede apreciar un ejemplo de un componente que accede al contexto del usuario actualmente autenticado (`currentUser`), a través del estado global de autenticación de *React Redux*.

```tsx
'use client';

import { useAppSelector } from '@/features/redux/redux-hook';
import { homeStyles } from '@/styles/app/home/page.styles';

export function HeaderHome() {
  const currentUser = useAppSelector(state => state.auth.currentUser);
  const styles = homeStyles;

  return (
    <header className={styles.header.wrapper}>
      <div className={styles.header.titleWrapper}>
        <h2 className={styles.header.title}>Hola, {currentUser?.fullName}</h2>
        <p className={styles.header.subtitle}>Explora comunidades y causas</p>
      </div>
    </header>
  );
}
```

Código 10: Contexto para obtener el usuario actualmente autenticado

3.4. Cliente de GraphQL con apollo

En este apartado, explicaremos nuestra solución del cliente de Apollo ya que incorpora muchos apartados destacables en este sentido de cara a cumplir el patrón DRY (Don't Repeat Yourself).

Aclarar que nuestra solución se apoya en la versión 4 del cliente de Apollo.

3.4.1. Configuración centralizada y entorno centrado en el polimorfismo

Para evitar la duplicidad de la lógica de conexión al servidor en diferentes partes de la aplicación web (Next.js), hemos diseñado el archivo de configuración `apollo-client.ts` respetando el polimorfismo. El cliente es capaz de inicializarse condicionalmente tanto en el lado del servidor (*Server-Side Rendering*) donde no existen WebSockets, como en el lado del cliente (*Client-Side Rendering*) sumando soporte para *Subscriptions*.

De este modo no necesitamos instanciar diversos clientes; disponemos de un único origen de verdad que expone el método adecuado dependiendo del entorno. Hemos integrado un gestor Singleton a través de `getInstanceServerApolloClient` para garantizar una única instancia viva en servidor.

```
export const createApolloClient = (): ApolloClient => {
  // Entorno Servidor: No definimos WebSocket Link
  if (globalThis.window === undefined) {
    return new ApolloClient({
      cache: new InMemoryCache(),
      link: ApolloLink.from([errorLink, httpLink]),
    });
  }

  // Entorno Cliente: Definimos WebSocket Link para Suscripciones
  const wsLink = new GraphQLWsLink(
    createClient({
      url: WS_URL,
      keepAlive: 15000,
      lazyCloseTimeout: 30000,
    })
  );

  const link = ApolloLink.split(
    ({ operationType }) => operationType === OperationTypeNode.SUBSCRIPTION,
    wsLink,
    httpLink
  );

  return new ApolloClient({
    cache: new InMemoryCache(),
    link: ApolloLink.from([errorLink, link]),
  });
};
```

Código 11: Inicialización unificada del Cliente Apollo (Singleton)

3.4.2. Gestión de errores transversal (ErrorLink)

Dentro de Apollo, uno de los mayores problemas vinculados al incumplimiento de DRY es el manejo manual de errores repitiendo bloques `try/catch` en cada petición. Hemos encapsulado un `ErrorLink` generalizado en el sistema de enlaces (*Links*) de Apollo que se inyecta globalmente. Intercepta cualquier excepción *GraphQLError* o error de red y lo formatea por `console.error` sin inyectar código extra al negocio.

```
const errorLink = new ErrorLink(({ error, operation }) => {
  const operationName = operation.operationName || 'anonymous';

  if (CombinedGraphQLErrors.is(error)) {
    error.errors.forEach(({ message }) => {
      console.error(`[GraphQL error][${operationName}] ${message}`);
    });
    return;
  }

  if (error) {
    const endpoint = operation.operationType === OperationTypeNode.SUBSCRIPTION
      ? WS_URL : HTTP_URL;
    console.error(`[Network error][${operationName}] ${error.message}`);
  }
});
```

Código 12: Interceptor de excepciones transversal de Apollo

3.4.3. Capa de Abstracción para Consultas (`GraphqlClientService`)

A pesar de que Apollo Client es una gran herramienta, las llamadas en crudo consumen mucho código *boilerplate*, es decir, código repetitivo. Para resolver esto y elevar la abstracción, hemos diseñado un servicio en `graphql-client.service.ts` inyectable, que procesa las consultas devolviendo directamente la información y garantizando el tipado fuerte (`TypedDocumentNode`) que se incorporó en la versión 4 de Apollo.

La lógica para las *queries* define por defecto `fetchPolicy: 'network-only'` para evitar comportamientos fantasma en caché no resueltos, y la lógica de desempaquetado de la respuesta (`response.data`) la hace el método de forma transparente en todas las mutaciones y lecturas.

```typescript
export class GraphqlClientService {
  constructor(private readonly client: GraphqlClient) {}

  // Tanto OperationVariables como TypedDocumentNode son nativos de Apollo 4
  async query<TData, TVariables extends OperationVariables>(
    document: TypedDocumentNode<TData, TVariables>,
    variables: TVariables
  ): Promise<TData> {
    const response = await this.client.query({
      query: document,
      variables,
      fetchPolicy: 'network-only',
    });

    if (!response.data) {
      throw new Error('GraphQL query returned no data');
    }

    return response.data;
  }

  async mutate<TData, TVariables extends OperationVariables>(
    document: TypedDocumentNode<TData, TVariables>, ...
  ): Promise<TData> {
    // Implementación análoga para mutaciones
    // extrayendo repetidamente ".data"
  }
}
```

Código 13: Servicio general de llamadas GraphQL y desempaquetado automático

El conjunto de estas tres prácticas (inicialización única polimórfica , control de errores generalizado, y envoltura adaptadora de la librería externa en un servicio fuertemente tipado), aseguran que en las operaciones del frontend solo deban definirse la *Query/-Mutación* deseada y sus variables; delegando resoluciones HTTP, tipados inferidos y excepciones completamente a la infraestructura común, satisfaciendo enormemente los principios DRY y SOLID.

3.4.4. Uso del cliente genérico en servidor (SSR) en Home

En servidor, la página de inicio construye el acceso a GraphQL con una única vía reutilizable: instancia compartida de Apollo para SSR + `GraphqlClientService`. Esto evita repetir configuración de transporte, deserialización de datos y control de errores por cada página del servidor.

```
const loadHomeData = async () => {
  const client = getInstanceServerApolloClient();
  const graphql = new GraphqlClientService(client);

  const [communitiesResult, causesResult] = await Promise.allSettled([
    graphql.query(GET_COMMUNITIES_QUERY, {}),
    graphql.query(GET_CAUSES_QUERY, {}),
  ]);

  const communityList =
    communitiesResult.status === 'fulfilled'
      ? communitiesResult.value.communities.slice(0, 3)
      : [];

  const causesList =
    causesResult.status === 'fulfilled'
      ? causesResult.value.causes.slice(0, 6)
      : [];

  return {
    communities: communityList,
    causes: causesList,
  };
};
```

Código 14: Uso SSR del cliente genérico en Home

Este patrón cumple DRY por tres motivos:

- Una sola factoría de cliente para servidor.
- Una sola API de consumo (`query/mutate/subscribe`) para todos los módulos.
- Una única política de validación de respuesta (`response.data`) dentro del servicio.

3.4.5. Uso en cliente con hooks de Apollo: `useQuery` y `useMutation`

En cliente, el árbol de la aplicación se envuelve una vez con `ApolloGraphqlProvider`, y desde ahí los componentes/hook consumen GraphQL sin reinstanciar clientes ni repetir configuración. Eso reduce el código repetitivo en cada feature.

```tsx
export default function SolidarianLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <Provider store={store}>
      <AuthInitializer>
        <ApolloGraphqlProvider>
          <NotificationSubscriptionsInitializer />
          <ThemeProvider>{children}</ThemeProvider>
        </ApolloGraphqlProvider>
      </AuthInitializer>
    </Provider>
  );
}
```

Código 15: Proveedor único de Apollo en cliente

Ejemplo de lectura con `useQuery` (hook de solicitudes de usuario):

```tsx
export function useUserRequest(userId?: string) {
  const currentUserId = useAppSelector(
    (state) => state.auth.currentUser?.id
  );
  const resolvedUserId = userId ?? currentUserId;

  const requestsQuery = useQuery(GET_REQUESTS_BY_USER_ID_QUERY, {
    variables: { userId: resolvedUserId ?? '' },
    skip: !resolvedUserId,
    fetchPolicy: 'network-only',
  });

  return {
    requests: requestsQuery.data?.getRequestsByUserId ?? [],
    loading: Boolean(resolvedUserId) ? requestsQuery.loading : false,
    error: Boolean(resolvedUserId)
      ? (requestsQuery.error?.message ?? null)
      : null,
  };
}
```

Código 16: Consulta en cliente con useQuery

Ejemplo de escritura con `useMutation` (seguir/dejar de seguir causa):

```
const [joinCause, { loading: joiningCause }] =
  useMutation(JOIN_CAUSE_MUTATION);
const [leaveCause, { loading: leavingCause }] =
  useMutation(LEAVE_CAUSE_MUTATION);

const handleFollow = async () => {
  const userId = currentUserId!;

  if (isFollowing) {
    const result = await leaveCause({ variables: { causeId, userId } });
    if (result.data?.leaveCause) setIsFollowing(false);
  } else {
    const result = await joinCause({ variables: { causeId, userId } });
    if (result.data?.joinCause) setIsFollowing(true);
  }
};
```

Código 17: Mutaciones en cliente con useMutation

Aquí DRY se logra porque los componentes solo expresan intención de negocio (leer o mutar), mientras la infraestructura (cliente, enlaces HTTP/WS, errores, caché y credenciales) ya está centralizada.

3.4.6. Suscripciones en ventana de notificaciones (`useSubscriptions`)

Para notificaciones en tiempo real no se usa `useSubscription` directamente en cada componente; se encapsula en un hook único (`useNotificationSubscriptions`) apoyado en `GraphqlClientService.subscribe`. Este diseño evita repetir la misma lógica de alta/baja de suscripciones en toda la UI.

```
export function NotificationSubscriptionsInitializer(): null {
  const user = useAppSelector((state) => state.auth.currentUser);

  useNotificationSubscriptions({
    userId: user?.id || '',
    enabled: !!user?.id,
  });

  return null;
}
```

Código 18: Inicialización global de suscripciones

```
const graphql = new GraphqlClientService(apolloClient);

const requestSubscription = graphql
  .subscribe(REQUEST_STATUS_CHANGED_SUBSCRIPTION, { userId })
  .subscribe({
    next: async (result) => {
      if (result.data?.requestStatusChanged) {
        const event = result.data.requestStatusChanged;
        if (
          event.status === 'APPROVED' ||
          event.status === 'REJECTED'
        ) {
          const notification = toRequestStatusNotification(event);
          await saveNotificationToDexie(notification);
        }
      }
    },
    error: (error: Error) => {
      console.error('requestStatusChanged subscription error:', error);
    },
  });
```

Código 19: Suscripción GraphQL encapsulada en hook de notificaciones

Finalmente, la ventana de notificaciones se actualiza de forma desacoplada escuchando un evento único (`NOTIFICATION_UPDATE_EVENT`), por lo que no hay que sincronizar manualmente cada componente de cabecera/listado:

```
useEffect(() => {
  const handleNotificationUpdate = () => {
    fetchNotifications();
  };

  window.addEventListener(
    NOTIFICATION_UPDATE_EVENT,
    handleNotificationUpdate
  );
  return () =>
    window.removeEventListener(
      NOTIFICATION_UPDATE_EVENT,
      handleNotificationUpdate
    );
}, [fetchNotifications]);
```

Código 20: Refresco de la ventana de notificaciones

En términos DRY, esta parte concentra en un solo lugar: 1) creación y destrucción de suscripciones, 2) transformación de eventos a entidad de dominio `Notification`, y 3) mecanismo de propagación a UI. Así se evita duplicar listeners, conversiones y lógica de ciclo de vida por pantalla.

3.5. Librerías de componentes utilizadas

En el *frontend* se consolidó una estrategia de UI basada en componentes reutilizables, accesibles y altamente personalizables. A partir del inventario real de *imports* del proyecto, las librerías de componentes adoptadas son las siguientes:

3.5.1. Shadcn/ui (núcleo de la interfaz).

Shadcn/ui no es una librería de componentes en el sentido convencional: en lugar de distribuirse como un paquete de `node_modules`, expone sus componentes como código fuente que se copia directamente al repositorio mediante su CLI. El proyecto fue creado gracias a su nueva función referenciada en el siguiente enlace [3]. Su filosofía de *ownership* total fue determinante para su elección por varios motivos:

1. **Propiedad y evolución sin fricción.** Al residir en el propio repositorio, cada componente puede modificarse, extenderse o descomponerse sin depender de una versión publicada por un tercero ni de *overrides* CSS frágiles.

2. **Integración nativa con Tailwind CSS y variables CSS.** Los componentes de *shadcn/ui* utilizan clases de *Tailwind* y variables CSS para la tematización (`-background`, `-foreground`, `-primary`, etc.), lo que permite cambiar el aspecto visual del proyecto de forma centralizada desde el fichero `globals.css`, sin tocar la lógica de los componentes.

3. **Accesibilidad garantizada por Base UI.** Internamente, *shadcn/ui* delega la gestión de interacciones accesibles (foco, ARIA, teclado) a *Base UI*, una de las librerías *headless* más rigurosas del ecosistema *React*. Esto cumple con los criterios WCAG sin coste de implementación adicional.

4. **Variedad de granularidad.** Cada componente se importa desde el directorio `components/ui`. En este proyecto se usa también un `index.ts` para facilitar la importación a gran escala. Algunos de los componentes más comunes que hemos utilizado son: `Button`, `Input`, `Textarea` (controles de formulario), `SidebarInset` y `SidebarProvider` (para el *layout* principal), `Separator` (separación visual), `Card`, `CardTitle` y `CardDescription` (bloques de contenido estructurado). Esta granularidad evita cargar código no utilizado y facilita el *tree-shaking*.

5. **Ecosistema activo y documentación de referencia.** *shadcn/ui* dispone de una comunidad muy activa y una documentación oficial detallada

6. **Consistencia visual sin imponer un sistema de diseño cerrado.** A diferencia de librerías como *Material UI* o *Ant Design*, *shadcn/ui* no impone una identidad visual particular.

3.5.2. Comparativa con React Bootstrap y justificación de descarte

React Bootstrap es una opción consolidada en el ecosistema *React*, sin embargo, su modelo arquitectónico no se alinea con los requisitos del proyecto. *Shadcn* proporciona componentes más personalizables donde es el desarrollador quien controla el estilo completamente a través de *Tailwind*. Además tiene un *bundle size* menor al importar solo los componentes que usamos en el proyecto, por lo que ofrece mejor *performance* sin dependencias pesadas. Por todo ello hemos elegido *Shadcn/ui* para adaptar los componentes a la identidad visual y requisitos específicos de *SolidarianID*.

3.6. PWA en SolidarianID

En este apartado se describe cómo se ha usado PWA en el proyecto. Para dicha implementación seguimos el tutorial oficial de PWA en *Next.js* [4]).

3.6.1. Fichero `manifest`

El primer paso fue crear el fichero `app/manifest.ts` mostrado en el trozo de código 21, el cual es esencial para que los dispositivos (como teléfonos Android o iOS) reconozcan la aplicación como instalable. Este archivo define el nombre (S*olidarianID*), los íconos (en 192x192 y 512x512 con propósito *maskable* para adaptarse a diferentes formas de íconos) y la configuración de visualización (en modo *standalone* para que se abra como *app* nativa, sin la interfaz del navegador). Gracias a esta información, los usuarios pueden agregar *SolidarianID* a su pantalla de inicio y lanzarla desde allí con los colores de tema y fondo configurados.

```ts
import type { MetadataRoute } from 'next';

export const APP_NAME = 'SolidarianID';
export const APP_DEFAULT_TITLE = 'SolidarianID';
export const APP_TITLE_TEMPLATE = '%s | SolidarianID';
export const APP_DESCRIPTION =
  'SolidarianID is a platform that connects volunteers with communities in need,
  ↪  fostering solidarity and collective action for a better world.';
export const APP_THEME_COLOR = '#ffffff';

export default function manifest(): MetadataRoute.Manifest {
  return {
    name: APP_NAME,
    short_name: APP_NAME,
    description: APP_DESCRIPTION,
    start_url: '/',
    display: 'standalone',
    background_color: '#ffffff',
    theme_color: '#000000',
    prefer_related_applications: false,
    icons: [
      {
        src: '/web-app-manifest-192x192.png',
        sizes: '192x192',
        type: 'image/png',
        purpose: 'maskable',
      },
      {
        src: '/web-app-manifest-512x512.png',
        sizes: '512x512',
        type: 'image/png',
        purpose: 'maskable',
      },
    ],
  };
}
```

Código 21: Fichero `manifest.ts`

3.6.2. Service worker, estrategias de caché y app shell

El archivo `sw.ts` 22 es el *Service Worker* de *SolidarianID*, un intermediario entre la aplicación *Next.js* y el navegador que permite que la PWA funcione *offline*, gestione cachés y reciba notificaciones *push*.

Para su implementación hemos utilizado la librería ***Serwist*** [2], que es la alternativa a *Workbox* recomendada por el equipo de *Next.js*, debido a una mejor integración con *Turbopack* (el *bundler* de nuestra *app*) y el uso nativo de *TypeScript*.

En nuestro *service worker* se definen cuatro estrategias de caché al crear el objeto `Serwist` (una explícita y tres implícitas en el objeto `defaultCache` de dicha librería):

1. ***NetworkOnly*** (explícita): Garantiza que peticiones no-GET que modifican el estado del servidor (POST, PUT, PATCH) nunca sean cacheadas, asegurando que

los datos siempre estén sincronizados con el *backend*.

2. ***CacheFirst*** (implícita en `defaultCache`): Para *assets* con *hash* (JS, CSS versionados). Intenta obtener los datos de la caché y, si esta falla, los obtiene de la red. Es seguro cachear indefinidamente porque cada cambio de código genera una URL única (nuevo *hash*), evitando colisiones de caché.

3. ***StaleWhileRevalidate*** (implícita en `defaultCache`): Para imágenes y fuentes externas (*Google Fonts*). Devuelve el contenido cacheado al instante (aunque esté obsoleto) mientras actualiza en segundo plano para la próxima visita, proporcionando excelente rendimiento sin sacrificar demasiada actualización de datos.

4. ***NetworkFirst*** (implícita en `defaultCache`): Para HTML, JSON y datos dinámicos. Intenta obtener los datos de la red primero para garantizar la frescura, pero utiliza caché como *fallback* si no hay conexión, proporcionando soporte en modo *offline*.

Adicionalmente, se implementa un *App Shell* básico mediante el `precacheEntries` de Serwist, que precachea de forma explícita todos los archivos críticos (layouts, bundles JS/CSS) para garantizar que la aplicación sea funcional en modo offline desde la primera visita.

Además, más adelante en el fichero `sw.ts` (aparece cortado por simpleza) se implementan manejadores de eventos que escuchan los eventos *push*, para mostrar notificaciones nativas del navegador cuando el servidor envía mensajes, con íconos personalizados y la opción *requireInteraction* para que el usuario deba interactuar con ellas. Al hacer clic en una notificación, el evento *notificationclick* la cierra y abre o enfoca la URL correspondiente según el tipo de notificación. El ciclo de vida está optimizado con *skipWaiting* y *clientsClaim* para que el nuevo *Service Worker* se active inmediatamente sin esperar a cerrar pestañas, y durante la activación se limpian cachés antiguos para evitar acumular basura. En conjunto, este `sw.ts` convierte *SolidarianID* en una PWA completa: rápida, funcional sin conexión para contenido estático, sincronizada con el servidor en datos sensibles y capaz de notificar a los usuarios incluso cuando la *app* no está abierta.

```
/// <reference no-default-lib="true" />
/// <reference lib="esnext" />
/// <reference lib="webworker" />

import { defaultCache } from '@serwist/turbopack/worker';
import type { PrecacheEntry, SerwistGlobalConfig } from 'serwist';
import { NetworkOnly, Serwist } from 'serwist';
import { PushNotificationPayload, StoredNotificationData } from
↪  '../features/pwa/types/payload.type';
import { getNotificationUrl } from '../features/pwa/utils';

declare global {
  interface WorkerGlobalScope extends SerwistGlobalConfig {
    __SW_MANIFEST: (PrecacheEntry | string)[] | undefined;
  }
}

declare const self: ServiceWorkerGlobalScope;

const serwist = new Serwist({
  precacheEntries: self.__SW_MANIFEST,
  skipWaiting: true,
  clientsClaim: true,
  navigationPreload: true,
  runtimeCaching: [
    {
      matcher({ request }) {
        return request.method !== 'GET';
      },
      handler: new NetworkOnly(),
    },
    ...defaultCache,
  ],
  fallbacks: {
    entries: [
      {
        url: '/~offline',
        matcher({ request }) {
          return request.destination === 'document';
        },
      },
    ],
  },
});

serwist.addEventListeners();

...
```

Código 22: Fichero `sw.ts`

3.6.3. Notificaciones push

Las notificaciones *web push* permiten que el servidor *backend* envíe mensajes directamente al navegador del usuario incluso cuando no está usando la aplicación. El flujo es: (1) Usuario se suscribe a notificaciones, (2) Se guarda su suscripción en el *backend*, (3) El

backend envía un mensaje a esa suscripción, (4) El *Service Worker* recibe el evento y lo maneja, de forma que lo muestra como notificación nativa del sistema.

En `features/pwa/utils.ts`, la función `subscribeToPush()` (23) hace tres cosas:

1. Obtiene el *Service Worker* registrado
2. Usa `pushManager.subscribe()` con la *VAPID public key* del servidor (convirtiendo el *base64* a *Uint8Array* con `urlBase64ToUint8Array()`)
3. Retorna un objeto `PushSubscription` con tres datos críticos: *endpoint* (URL donde enviar la notificación al proveedor *push*), *p256dh* (clave para encriptar el *payload*) y *auth* (token de autenticación)

Estas tres propiedades son lo que identifica al usuario para recibir *push*.

```
export async function subscribeToPush(): Promise<PushSubscription | null> {
  const registration = await globalThis.navigator.serviceWorker.ready;
  const vapidPublicKey = process.env.NEXT_PUBLIC_VAPID_PUBLIC_KEY;

  if (!vapidPublicKey) {
    throw new Error('Missing NEXT_PUBLIC_VAPID_PUBLIC_KEY environment variable.');
  }

  const sub = await registration.pushManager.subscribe({
    userVisibleOnly: true,
    applicationServerKey: urlBase64ToUint8Array(vapidPublicKey),
  });

  return sub;
}
```

Código 23: Función `subscribeToPush()` de `features/pwa/utils.ts`

Por otro lado, el *hook* `usePwa()` controla el ciclo de vida. Su función `subscribe()` (24) primero obtiene la suscripción actual (o crea una nueva con `subscribeToPush()` que acabamos de explicar) y luego, con el método `registerSubscriptionInBackend()` se codifican las claves *p256dh* y *auth* en *base64*, para finalmente enviar un POST a `/api/subscriptions` con el *endpoint*, *p256dh* y *auth*. De esta forma, el *backend* guarda dicha información en base de datos; sin esto, el *backend* no sabría a quién enviar notificaciones.

```
async function subscribe() {
    if (!isSupported) {
      throw new Error('Push notifications are not supported in this browser.');
    }

    const currentSubscription = await getCurrentPushSubscription();
    const sub = currentSubscription ?? (await subscribeToPush());

    if (!sub) {
      throw new Error('No se pudo crear la suscripción push.');
    }

    await registerSubscriptionInBackend(sub);
    setSubscription(sub);
}
```

Código 24: Función `subscribe()` de `hooks/pwa/use-pwa.ts`

De esta forma, cuando el *backend* envía una notificación *push* (mediante la librería *web-push*), el *Service Worker* la recibe en el evento *push*, tal como se muestra en el trozo de código 25. Esta función manejadora parsea el JSON del *payload*, extrae el tipo de notificación y datos (como *requestId*, *communityId*), y construye un objeto `NotificationOptions` que incluye diversos campos como título, mensaje, icono, *badge* y los datos. Luego llama a `self.registration.showNotification()` para mostrar una notificación nativa del sistema operativo.

```
self.addEventListener('push', (event: PushEvent) => {
  if (!event.data) {
    return;
  }

  const payload = event.data.json() as PushNotificationPayload;

  const notificationData: StoredNotificationData = {
    ...(payload.data ?? {}),
    type: payload.type,
  };

  const options: NotificationOptions = {
    body: payload.message,
    tag: payload.type,
    icon: '/solidarian-logo.png',
    badge: '/badge.png',
    data: notificationData,
    requireInteraction: true,
  };

  event.waitUntil(self.registration.showNotification(payload.title, options));
});
```

Código 25: Manejador de *push* en `app/sw.ts`

Cuando el usuario hace clic en la notificación, se dispara el evento *notificationclick*, el cual es manejado por la función mostrada en el trozo de código 26. El código extrae los

datos guardados en la notificación (tipo, *requestId*, etc.), cierra la notificación, y usa la función `getNotificationUrl()` (en `utils.ts`) para determinar a dónde navegar: si es tipo `ACTION` va a la acción que se acaba de crear, si es `CAUSE` va a la causa que se ha creado y si es `REQUEST_STATUS_CHANGED` va a la solicitud que se acaba de aceptar o rechazar. Finalmente, `openOrFocusWindow()` abre o enfoca esa URL en la *app*.

```
self.addEventListener('notificationclick', (event: NotificationEvent) => {
  const notification = event.notification;
  const data = notification.data as StoredNotificationData | undefined;

  notification.close();

  const urlToOpen = getNotificationUrl(data);

  event.waitUntil(openOrFocusWindow(urlToOpen));
});
```

Código 26: Manejador de *notificationclick* en `app/sw.ts`

3.7. Almacenamiento local basado en Dexie

Si bien es cierto que *IndexedDB* es la API nativa del navegador para almacenamiento local estructurado (clave-valor, índices, transacciones), nosotros hemos utilizado la librería *Dexie*, ya que esta encapsula *IndexedDB* para hacerla más fácil de usar (sintaxis simplificada, manejo de promesas, etc.).

En nuestro *frontend* de *SolidarianID*, usamos *Dexie* para almacenar notificaciones localmente que llegan vía *GraphQL subscriptions*. El flujo es: el *backend* envía eventos en tiempo real (nueva causa, nueva acción, cambio de estado de solicitud), los recibimos por *GraphQL* en el *frontend*, los guardamos en la BD local de *Dexie*, y la *app* los muestra sin depender del servidor. Esto permite que las notificaciones persistan incluso si cierras la *app*, se desconecta la red o el usuario cambia de página. En los siguientes subapartados se detalla este flujo con ejemplos de código.

3.7.1. Recibir y guardar notificaciones en Dexie

El *hook* `useNotificationSubscriptions` se suscribe a tres eventos *GraphQL* que llegan desde el *backend*: `REQUEST_STATUS_CHANGED, CAUSE_CREATED, ACTION_CREATED`. En el trozo de código 27 se muestra el ejemplo de una suscripción *graphql* al evento llamado `REQUEST_STATUS_CHANGED`, que se dispara cuando una solicitud es aceptada o rechazada por un administrador de la plataforma.

```
const requestSubscription = graphql.subscribe(REQUEST_STATUS_CHANGED_SUBSCRIPTION, {
↪  userId }).subscribe({
    next: async (result: { data: RequestStatusChangedResult | undefined }) => {
      if (result.data?.requestStatusChanged) {
        const event = result.data.requestStatusChanged;

        if (event.status === 'APPROVED' || event.status === 'REJECTED') {
          const notification = toRequestStatusNotification(event);
          await saveNotificationToDexie(notification);
        }
      }
    },
    error: (error: Error) => {
      console.error('requestStatusChanged subscription error:', error);
    },
  });
```

Código 27: Suscripción *graphql* a evento `REQUEST_STATUS_CHANGED`

Como se puede observar, cuando llega un evento se convierte a notificación y se guarda en el almacenamiento local de *Dexie* usando `notificationsRepository.save()`, tal como se muestra en el trozo de código 28.

```
const saveNotificationToDexie = useCallback(async (notification: Notification):
↪  Promise<void> => {
  await notificationsRepository.save(notification);
  globalThis.dispatchEvent(new Event(NOTIFICATION_UPDATE_EVENT));
}, []);
```

Código 28: Almacenamiento de una notificación en Dexie

Finalmente se dispara el evento `NOTIFICATION_UPDATE_EVENT` para avisar al componente `NotificationsPage` que hay cambios, el cual estará escuchando gracias a la función `window.addEventListener()` contenida en su `useEffect()`, tal como muestra el trozo de código 29. Una vez se recibe este evento, se vuelven a leer las notificaciones del almacenamiento de *Dexie* usando `fetchNotifications()` y se refresca el contenido, tal como veremos en el siguiente apartado.

```
useEffect(() => {
    const handleNotificationUpdate = () => {
      fetchNotifications();
    };

    window.addEventListener(NOTIFICATION_UPDATE_EVENT, handleNotificationUpdate);

    return () => {
      window.removeEventListener(NOTIFICATION_UPDATE_EVENT, handleNotificationUpdate);
    };
  }, [fetchNotifications]);

  const handleMarkAllAsRead = useCallback(() => {
    fetchNotifications();
  }, [fetchNotifications]);
```

Código 29: Página de notificaciones escuchando el evento `NOTIFICATION_UPDATE_EVENT`

3.7.2. Leer notificaciones de Dexie

Tal y como se ha comentado en el apartado anterior, después de escuchar el evento `NOTIFICATION_UPDATE_EVENT` se llama a `fetchNotifications()` para refrescar la página `/notifications`. Este método `fetchNotifications()` no consulta el servidor, sino que consulta *Dexie* directamente para leer las notificaciones de su almacenamiento, utilizando paginación, filtrado por el usuario actualmente autenticado y ordenamiento por fecha para que primero aparezcan las más recientes, tal como se muestra en el trozo de código 30.

```
const fetchNotifications = useCallback(
    async (pageToLoad = page) => {
      if (!currentUserId) {
        setNotifications({
          items: [],
          total: 0,
          offset: 0,
          limit: PAGE_SIZE,
        });
        setLoading(false);
        return;
      }

      try {
        const data = await notificationsRepository.paginate({
          filter: { userId: currentUserId },
          sortBy: 'createdAt',
          sortDirection: 'desc',
          offset: (pageToLoad - 1) * PAGE_SIZE,
          limit: PAGE_SIZE,
        });
        setNotifications(data);
      } catch (error) {
        console.error('Error fetching notifications:', error);
      } finally {
        setLoading(false);
      }
    },
    [currentUserId, page]
  );
```

Código 30: Cargar notificaciones del almacenamiento de *Dexie* en la página

4. Conclusiones

El desarrollo de esta práctica ha consolidado una visión integral del *Full-Stack* aplicada a un caso de estudio realista: *SolidarianID*. A lo largo del trabajo hemos diseñado e implementado una solución completa que combina una aplicación principal basada en microservicios con una aplicación *frontend* que facilita la interacción con los usuarios finales.

En el ámbito del ***frontend***, el desarrollo de la aplicación con ***Next.js*** nos ha servido para familiarizarnos con la herramienta y ver su comportamiento en un entorno académico, al igual que con el resto del *stack* tecnológico: *Shadcn/ui* como la librería de componentes recomendada para *Next.js* y *Tailwind CSS* para el uso ágil de estilos.

A nivel de **ingeniería del software**, este trabajo ha reforzado habilidades fundamentales: la capacidad de justificar decisiones técnicas, el diseño orientado a extensibilidad y la organización del equipo en un proyecto de desarrollo.

Como líneas de mejora, el proyecto queda preparado para seguir ampliando funcionalidades del dominio, robustecer mecanismos de integración entre servicios, y continuar

elevando el nivel de calidad mediante mayor automatización (por ejemplo, ampliación de pruebas y validaciones) y refuerzo de aspectos no funcionales conforme el sistema evolucione. En definitiva, *SolidarianID* ha servido como un entorno práctico para consolidar conocimientos técnicos y metodológicos propios del desarrollo *Full-Stack* moderno, con una base arquitectónica sólida para iterar de forma segura en futuras entregas cuando incorporemos los microservicios de gestión de red social y de mensajería entre usuarios.

Referencias

[1] Equipo 3. *Repositorio de GitHub del equipo 3*. 2026. URL: https://github.com/fsoladur/SolidarianID.

[2] Serwist. *Serwist*. 2026. URL: https://serwist.pages.dev/docs/next.

[3] Vercel. *Creador de proyectos de Shadcn*. 2026. URL: https://ui.shadcn.com/create.

[4] Vercel. *Tutorial de PWA para Next.js*. 2026. URL: https://nextjs.org/docs/app/guides/progressive-web-apps.

A. Nuestra interfaz

A.1. Páginas de la interfaz

A continuación, se verán todas las páginas de nuestra interfaz.

Landing page



Figura 3: Landing page

En esta imagen se puede ver la página de bienvenida de la aplicación, con una presentación general de la plataforma y accesos directos para iniciar sesión o registrarse.

Inicio de sesión

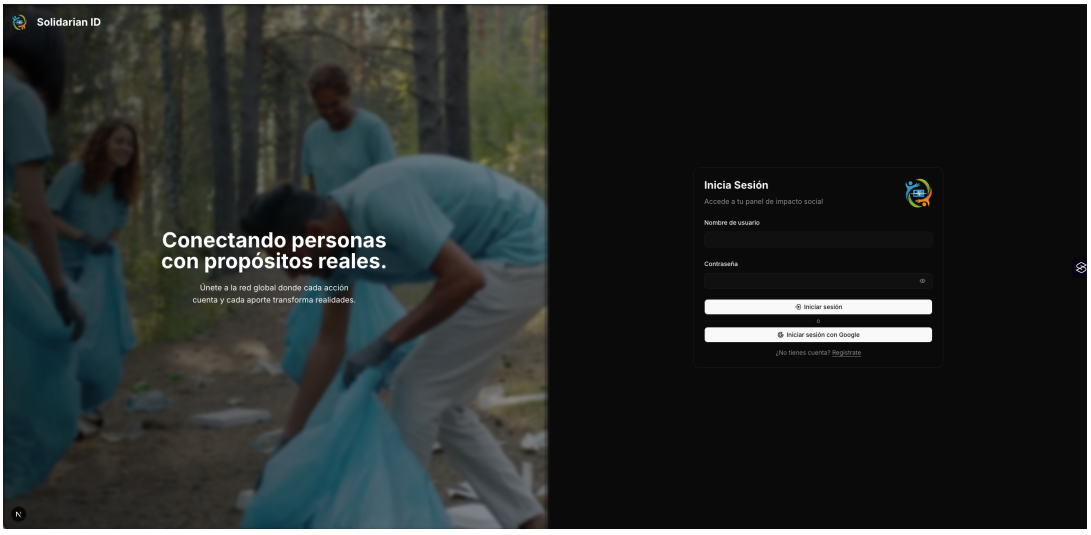


Figura 4: Inicio de sesión

En esta imagen se puede ver el formulario de inicio de sesión con campos para el nombre de usuario y la contraseña, así como la opción de autenticación mediante Google.

Registro de usuarios

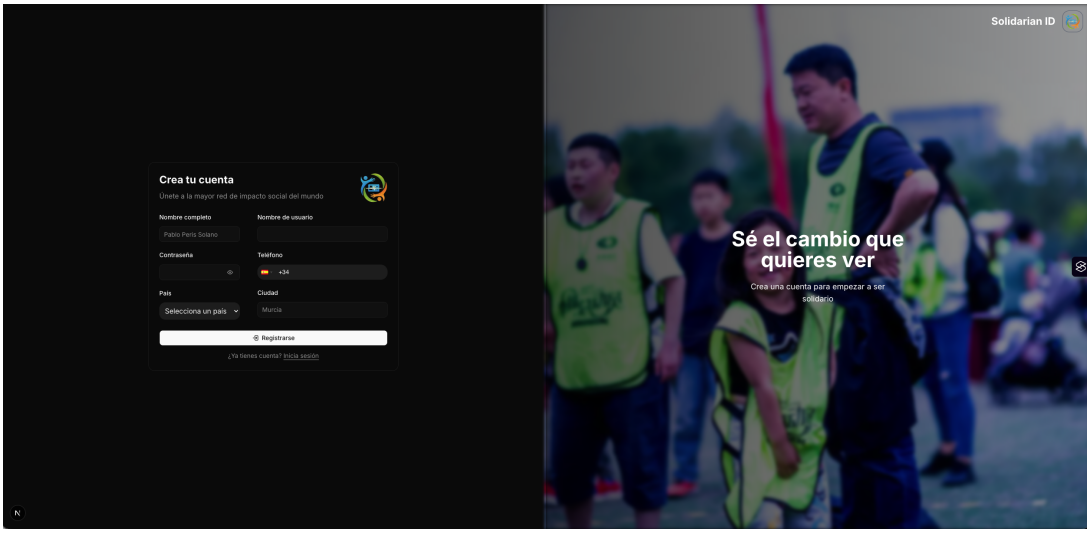


Figura 5: Registro de usuarios

En esta imagen se puede ver el formulario de registro con campos para el nombre completo, nombre de usuario, teléfono, contraseña, país y ciudad.

Preferencias

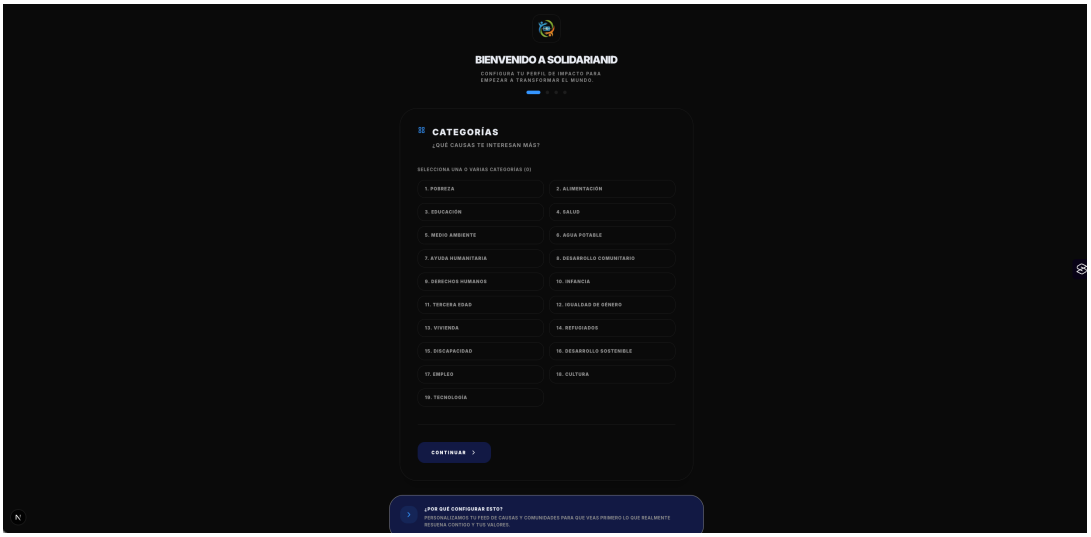


Figura 6: Preferencias

En esta imagen se puede ver el primer paso del asistente de selección de preferencias que se muestra al usuario tras su primer acceso a la aplicación.

Página de inicio

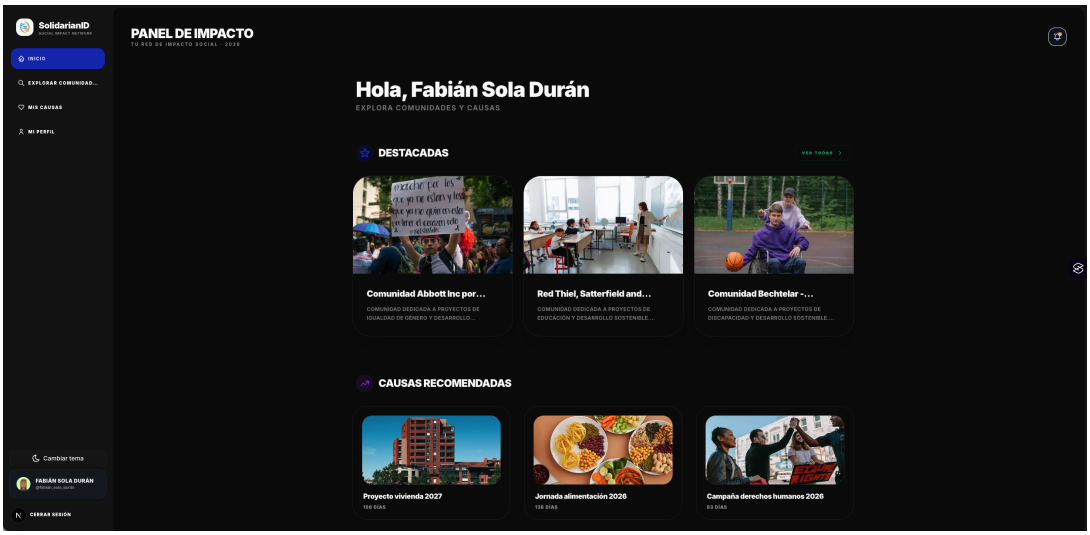


Figura 7: Home page

En esta imagen se puede ver la página principal de la aplicación con un resumen de las comunidades, causas y acciones relevantes para el usuario.

Explorar comunidades y causas

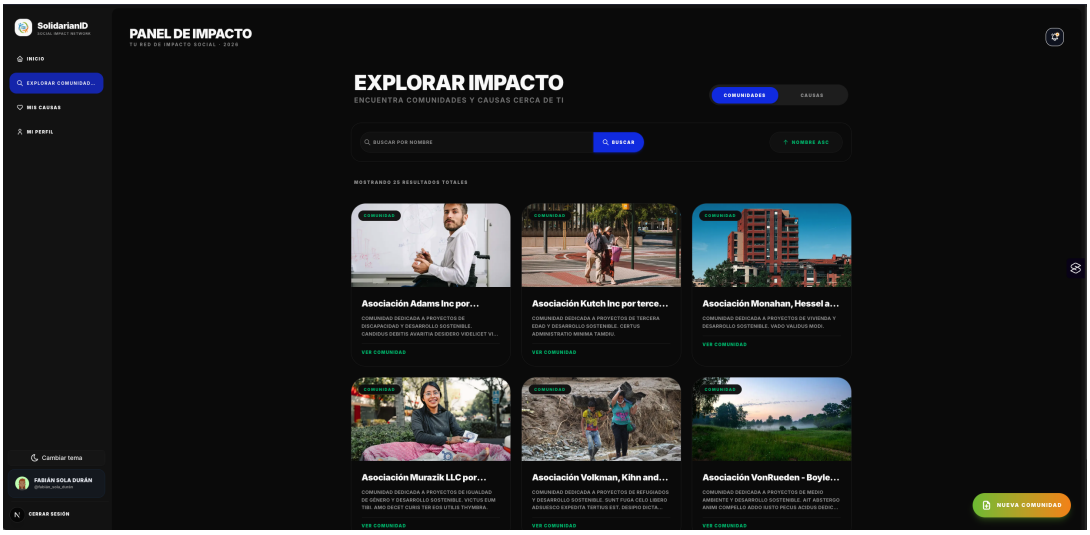


Figura 8: Explorar comunidades y causas

En esta imagen se puede ver el catálogo de comunidades y causas disponibles en la plataforma, con opciones de búsqueda y filtrado.

Crear solicitud de comunidad

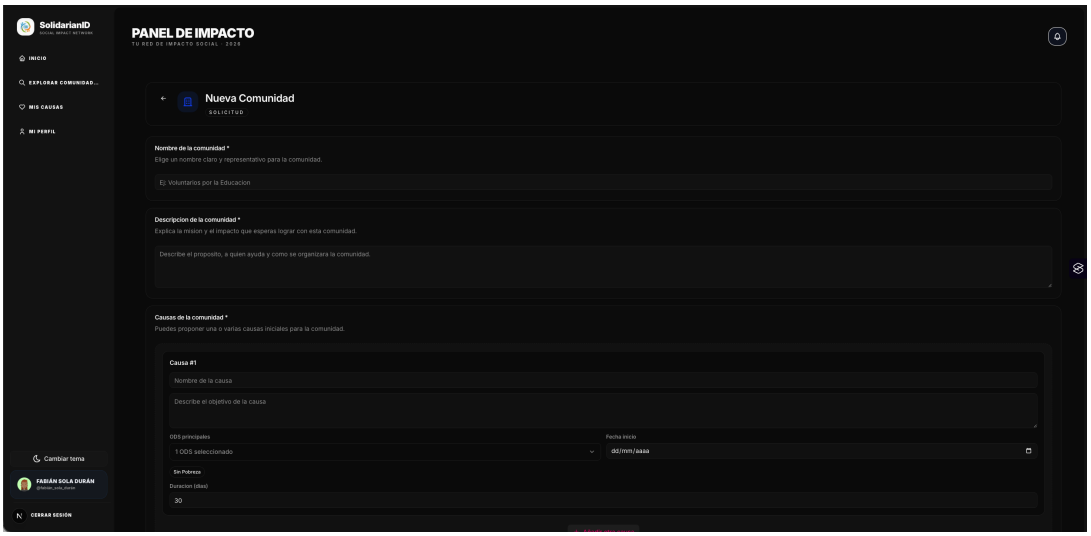


Figura 9: Formulario de creación de la comunidad

En esta imagen se puede ver el formulario para solicitar la creación de una nueva comunidad, con los campos necesarios para definirla.

Crear solicitud de causa

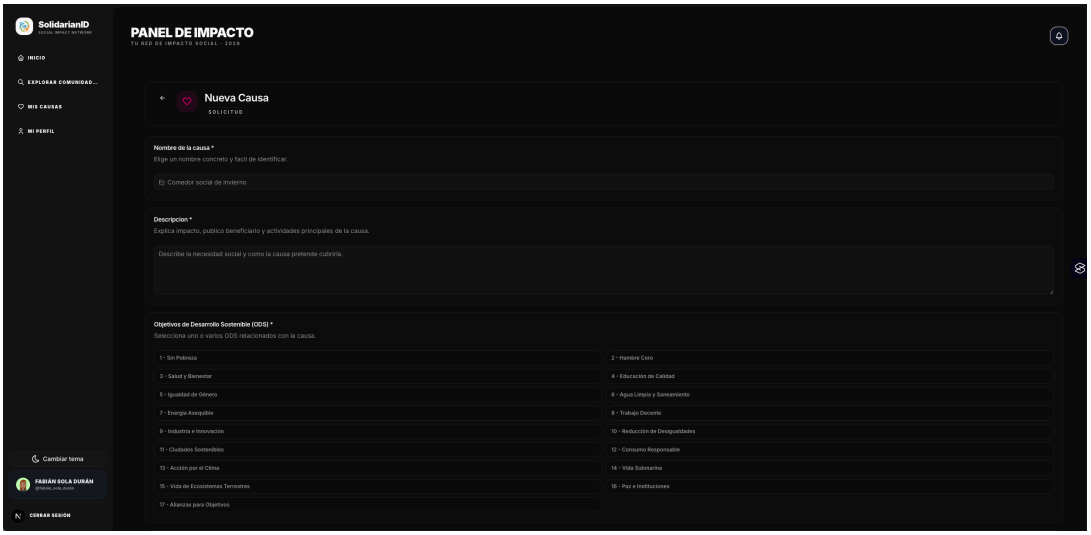


Figura 10: Formulario de creación de una causa

En esta imagen se puede ver el formulario para solicitar la creación de una nueva causa dentro de una comunidad existente.

Crear solicitud de acción

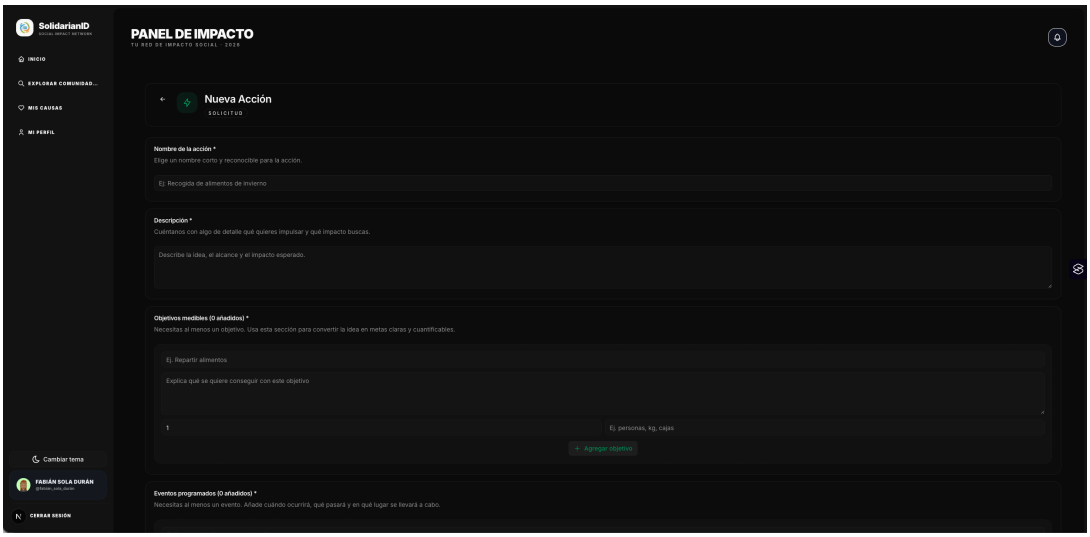


Figura 11: Formulario de creación de una acción

En esta imagen se puede ver el formulario para solicitar la creación de una acción asociada a una causa, con sus campos descriptivos correspondientes.

Detalle de comunidad

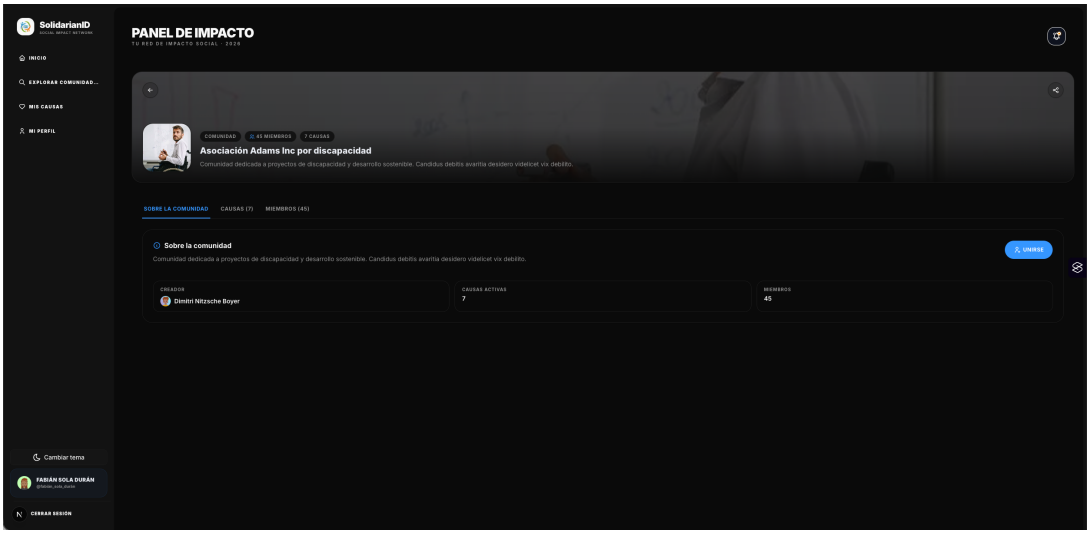


Figura 12: Detalle de comunidad

En esta imagen se puede ver la vista detallada de una comunidad, con su información, causas asociadas y opciones de participación.

Detalle de causa

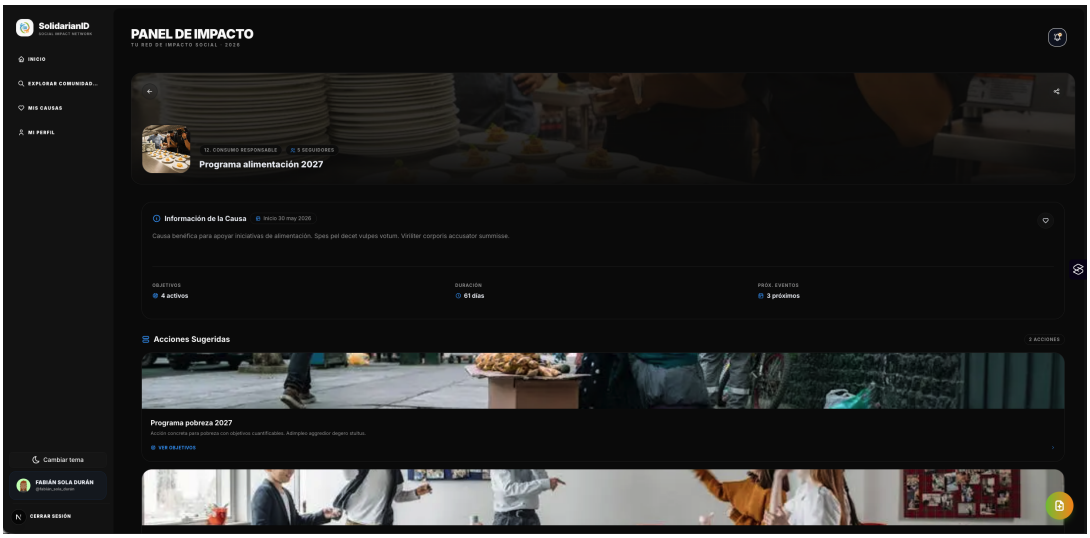


Figura 13: Detalle de causa

En esta imagen se puede ver la vista detallada de una causa, con su descripción, acciones vinculadas y opciones para unirse o apoyarla.

Detalle de acción

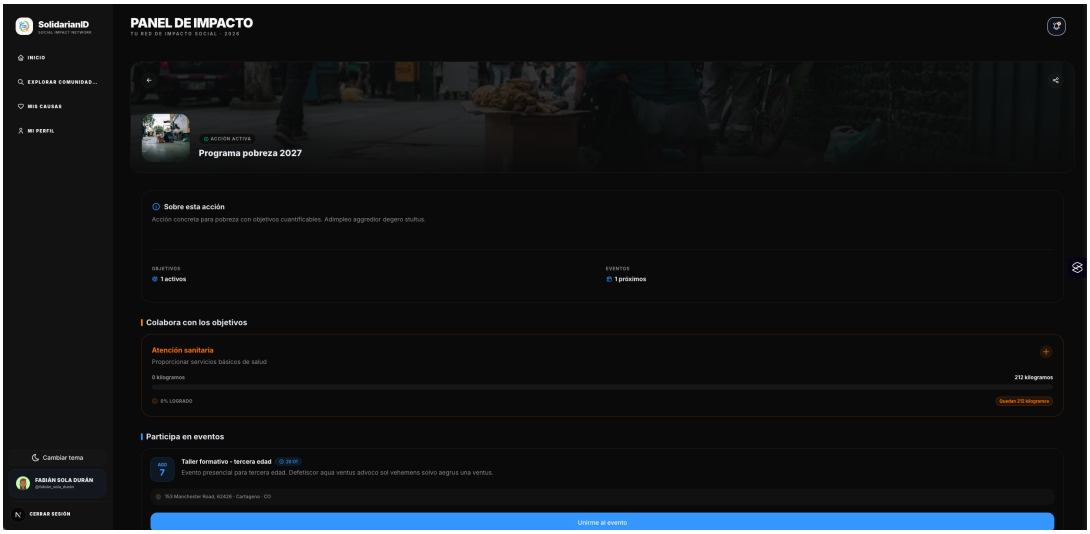


Figura 14: Detalle de acción

En esta imagen se puede ver la vista detallada de una acción concreta, con su información, estado y opciones de participación disponibles.

Mis causas

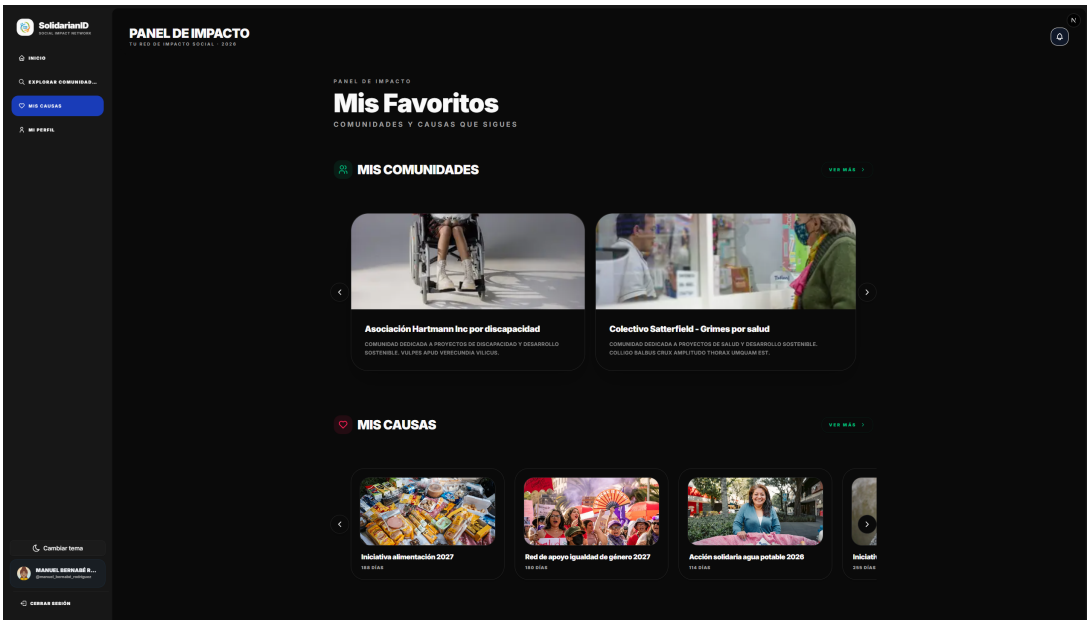


Figura 15: Mis causas

En esta imagen se puede ver el listado de causas en las que el usuario autenticado participa o sigue activamente.

Notificaciones del usuario

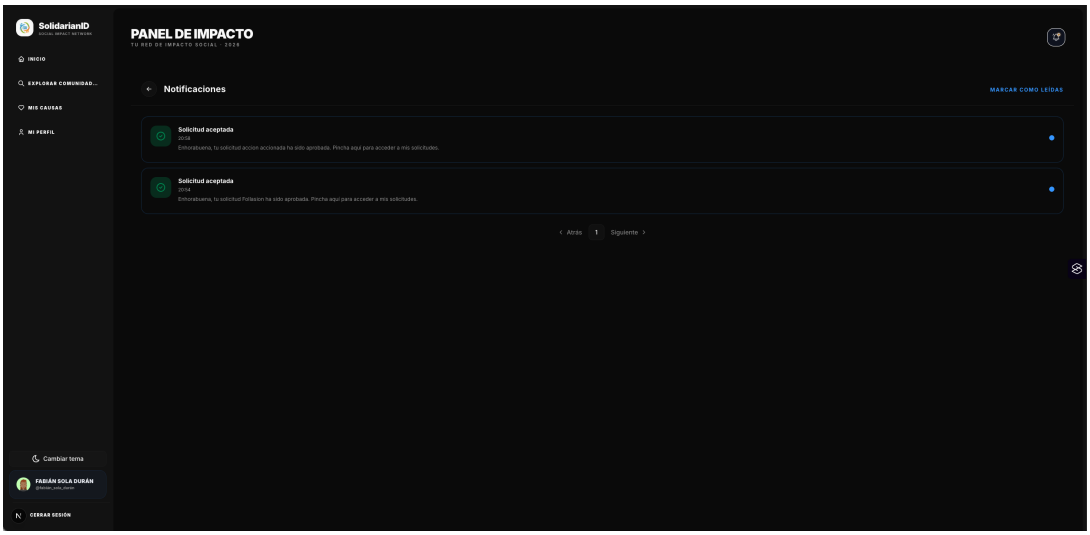


Figura 16: Notificaciones del usuario

En esta imagen se puede ver el panel de notificaciones del usuario con los avisos recientes relacionados con su actividad en la plataforma.

Mi perfil

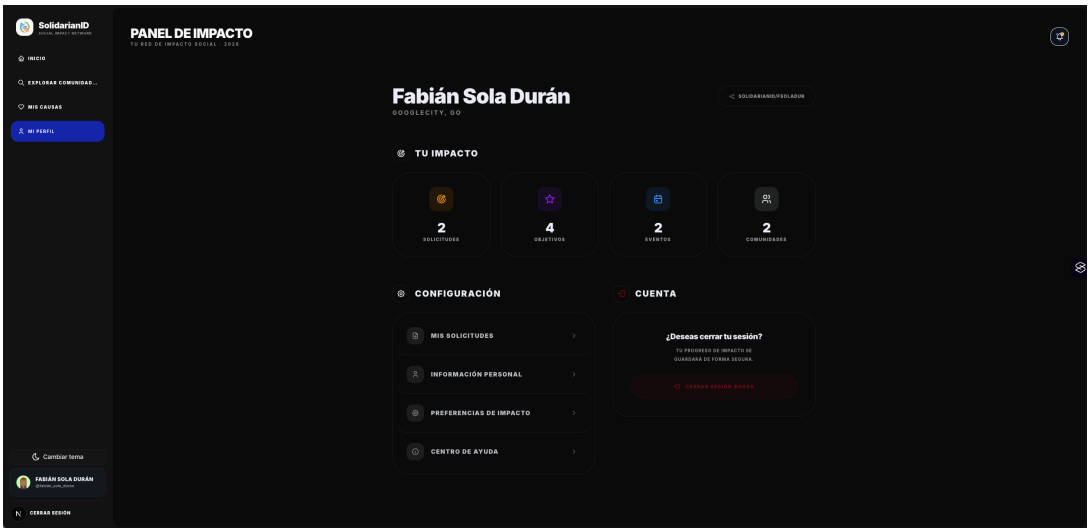


Figura 17: Mi perfil

En esta imagen se puede ver la página de perfil del usuario con su información pública, comunidades a las que pertenece y actividad reciente.

Mis solicitudes

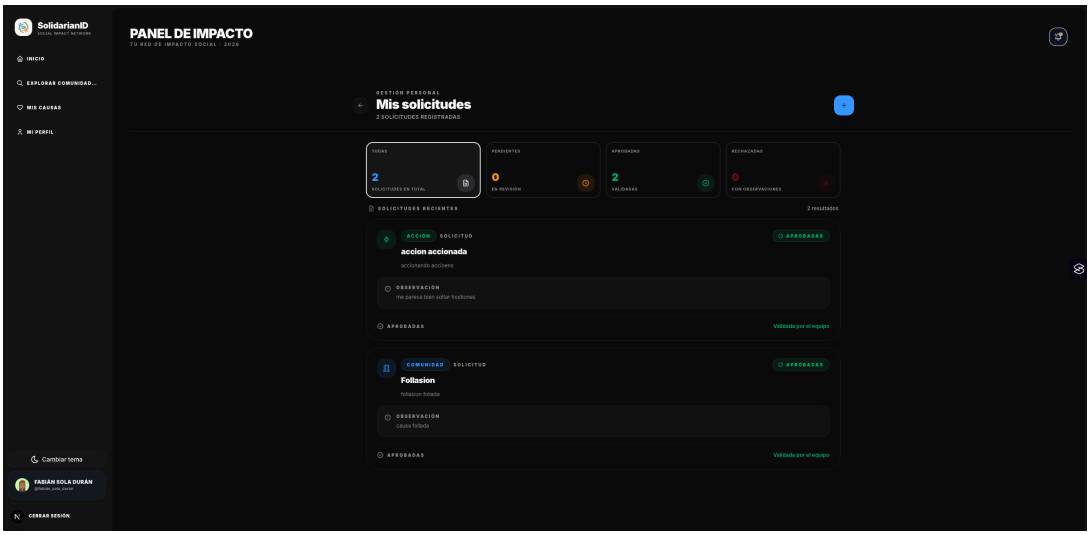


Figura 18: Mis solicitudes

En esta imagen se puede ver el historial de solicitudes enviadas por el usuario, con su estado actual (pendiente, aprobada o rechazada).

Información personal del perfil

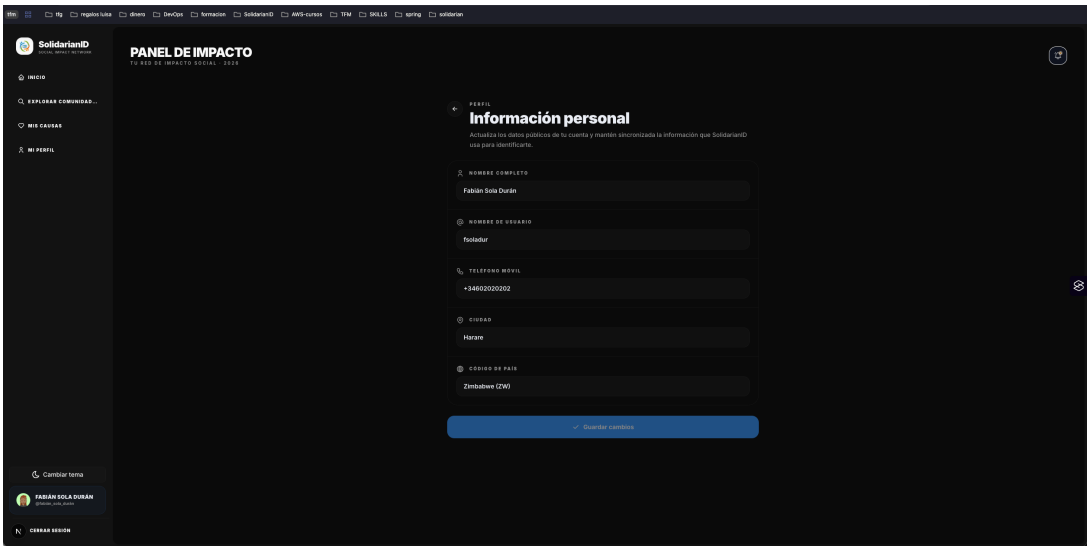


Figura 19: Mi información personal

En esta imagen se puede ver el formulario de edición de la información personal del usuario, como nombre, teléfono, país y ciudad.

Centro de ayuda

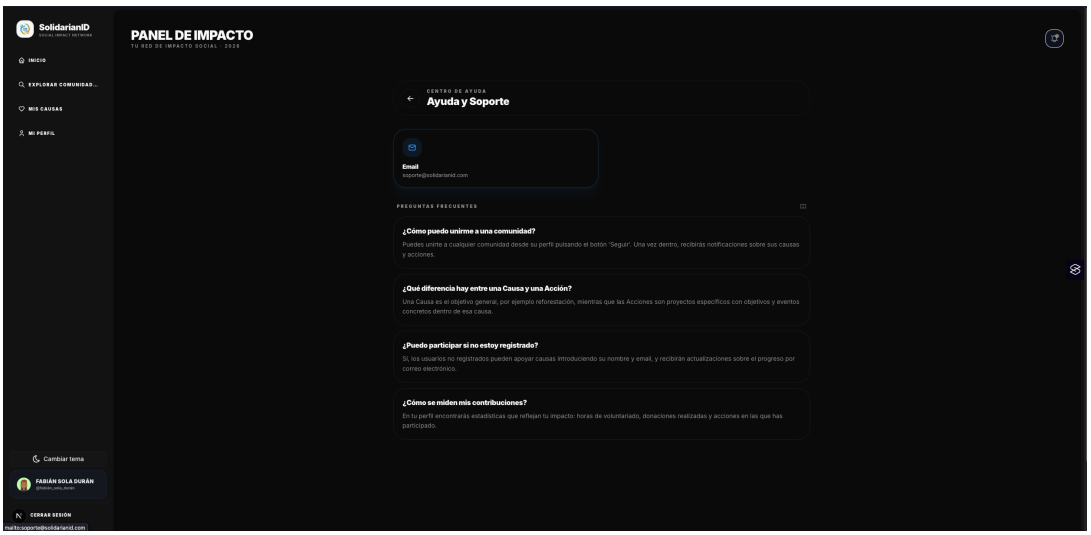


Figura 20: Centro de soporte y ayuda

En esta imagen se puede ver el centro de ayuda de la aplicación, con preguntas frecuentes y opciones de contacto con el soporte.

Mis favoritos

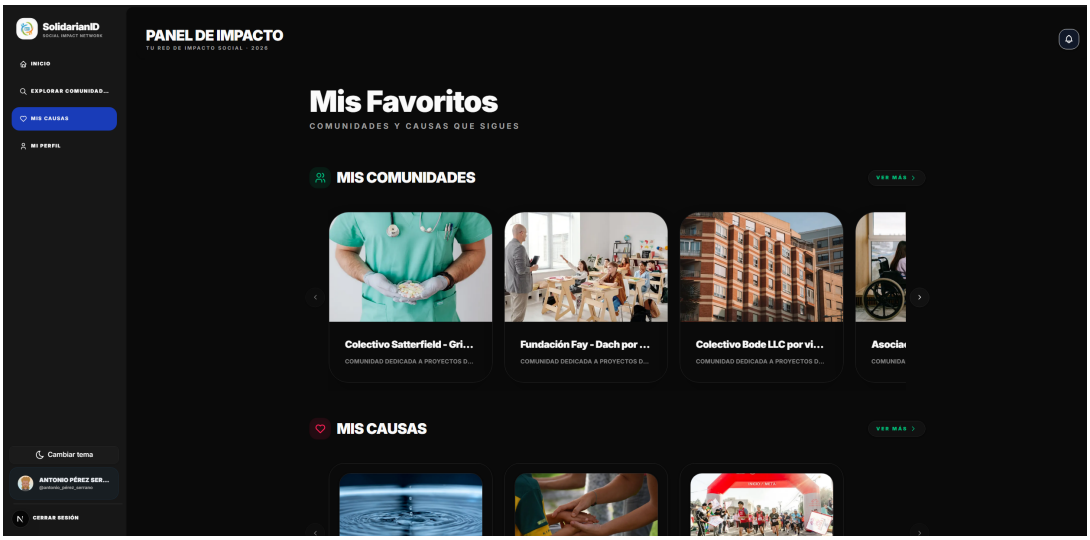


Figura 21: Mis favoritos

En esta imagen se puede ver la sección de favoritos del usuario, donde se puede ver un carrusel para las comunidades a las que pertenece el usuario y las causas que sigue, y un botón para abrir un modal para verlas con más detalle.

A.2. Componentes interesantes de la interfaz

Botón de compartir

Este botón nos permite compartir comunidades, causas o acciones a través de cualquier opción de mensajería.



Figura 22: Botón de compartir